Exercise 5 – Feature Effects

Interpretable Machine Learning

Hint: Useful libraries

R

```
library(ggplot2)
library(randomForest)
library(reshape2)
```

Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import ParameterGrid
```

Exercise 1: PDP and ICE in case of interactions

Learning goals

1. Derive PD and ICE functions analytically for a model with an interaction term.
2. Explain why PDPs can obfuscate heterogeneous effects and why ICE curves should be displayed alongside them.
3. Implement ICE, PDP and their centered variants from scratch.

Let us assume that we fitted the following linear regression model with two features:

$$\hat{f}(\mathbf{x}) = \hat{f}(x_1, x_2) = \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \hat{\beta}_3 x_1 x_2 + \hat{\beta}_0.$$

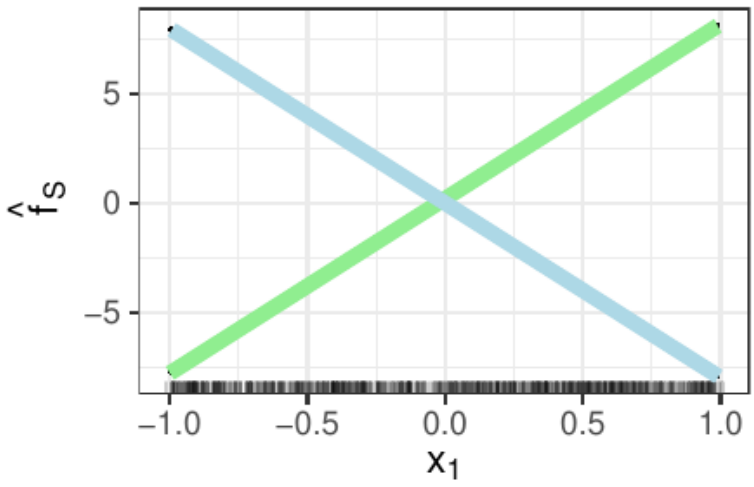
a) Analytically derive the PD function of feature $S = \{1\}$ (with $C = \{2\}$).

Hint

In the lecture slides we derived the PD function for a linear regression model without an interaction term. In the end, your solution should include terms like the expected value of X_2.

b) Let us assume that the estimated coefficients in the model above are $\hat{\beta}_0 = 0$, $\hat{\beta}_1 = -8$, $\hat{\beta}_2 = 0.2$ and $\hat{\beta}_3 = 16$. Furthermore, $X_1 \sim Unif(-1, 1)$ is uniformly distributed between -1 and 1, and $X_2 \sim B(1, 0.5)$ origins from a Bernoulli distribution. Compute the exact PD function of feature X_1 using your derived function of a).

c) The following plot displays the ICE curves of X_1. The rugs show the marginal distribution of X_1. Please note that since X_2 is binary, we do not receive n individual ICE curves, but indeed only two unique ones. Derive the conditional expectation functions $\hat{f}_1^{(i)}(x_1)$ of X_1 for group $X_2 = 1$ and for group $X_2 = 0$ using the model above given the estimated coefficients of b). Which color coding (light green or light blue) reflects the group $X_2 = 1$, which one group $X_2 = 0$?



d) Add the PDP you derived in b) to the plot. Use this example to explain briefly why it is advisable to display not only the PDP but also the ICE curves, when visualizing the feature effects of a specific model.

e) **Implementation: ICE and PDP.** Write functions that, given a fitted model object, a data matrix $\mathbf{X}$, a feature index j and a specified grid of values, do the following:

   (i) `ice(model, X, j, grid)` – returns an $n \times g$ matrix of ICE predictions (one curve per row, one grid value per column);
   (ii) `pdp(model, X, j, grid)` – returns the length-g PD vector by averaging the ICE matrix column-wise.

f) **Implementation: centered variants.** Extend the code with

   (i) `c_ice(model, X, j, grid, ref = 1)` – centers every ICE curve by subtracting its value at the reference grid index `ref` (the "vertical shift" in the slides which is often set to the minimum value);
   (ii) `c_pdp(model, X, j, grid, ref = 1)` – averages the centered ICE curves to obtain a centered PDP which is zero at the reference point.

   Use the definitions of c-ICE and c-PDP from the lecture slides to guide your implementation.

Exercise 2: Derivative, Forward, and Non-Linearity Effects

Learning goals

1. Derive derivative and forward marginal effects analytically.
2. Explain how curvature makes dME and fME differ for finite step sizes.
3. Compute and interpret the Non-Linearity Measure (NLM) as an R^2 of the local secant.

Let us consider the following non-linear regression model:

$$\hat{f}(x_1, x_2) = \theta_0 + \theta_1 x_1^2 + \theta_2 x_2^2 + \theta_{1,2} x_1 x_2$$

with parameters $\theta_0 = 0$, $\theta_1 = 1$, $\theta_2 = 0.5$, and $\theta_{1,2} = 2$.

a) Derive the analytic expression for the *derivative marginal effect* (dME) of feature x_1.

b) Derive the expression for the *forward marginal effect* (fME) of x_1 with step size $h_1 > 0$.

c) Let $x_1 = 1$, $x_2 = 2$, and $h_1 = 1$. Compute $\text{dME}_1(1, 2)$ and $\text{fME}_1(1, 2; h_1 = 1)$ and explain why these two results differ and interpret the additional terms appearing in the fME.

d) **Non-Linearity Measure (NLM):** We want to test how well the secant (local linear approximation) fits the model output along the direction of x_1. The NLM defined below quantifies how well the local linear approximation (secant) explains the variation in the model output along the direction of h_1. For T grid points $(\mathbf{x}^{(i)})_{i=1}^T$, it is defined as:

$$\text{NLM} = 1 - \frac{\overbrace{\sum_{i=1}^{T} \left[\hat{f}(\mathbf{x}^{(i)}) - g(t_i)\right]^2}^{\text{SSR: error of secant}}}{\underbrace{\sum_{i=1}^{T} \left[\hat{f}(\mathbf{x}^{(i)}) - \bar{f}\right]^2}_{\text{SST: total variance}}}.$$

- $t_i \in (0, 1)$ parametrises the path (here only in x_1 direction): $\mathbf{x}^{(i)} = (x_1 + t_i h_1, x_2)$.
- $g(t_i) = \hat{f}(x_1, x_2) + t_i \cdot \text{fME}_1(x_1, x_2; h_1)$ is the secant (straight line) through the two end-points.
- $\bar{f} = \frac{1}{T} \sum_{i=1}^{T} \hat{f}(\mathbf{x}^{(i)})$ is the mean prediction on the path.
- Hence NLM is an R^2: 1 means perfect linear fit, values $\ll 1$ reveal curvature.

Compute the NLM for $x_1 = 1$, $x_2 = 2$, and stepsize $h_1 = 0.5$. Interpret your result in relation to the fit of a local linear approximation using the $T = 9$ equidistant points $t_i = 0.1, 0.2, \ldots, 0.9$ below:

i	t_i	$x_1^{(i)} = 1 + 0.5t_i$	$\hat{f}(\mathbf{x}^{(i)})$	$g(t_i)$
1	0.1	1.05	7.3025	7.3250
2	0.2	1.10	7.6100	7.6500
3	0.3	1.15	7.9225	7.9750
4	0.4	1.20	8.2400	8.3000
5	0.5	1.25	8.5625	8.6250
6	0.6	1.30	8.8900	8.9500
7	0.7	1.35	9.2225	9.2750
8	0.8	1.40	9.5600	9.6000
9	0.9	1.45	9.9025	9.9250

e) **Implementation:** Write three functions:

- A function `dME(f, x, j)` that computes a central difference approximation of the derivative marginal effect for feature x_j at input x.
- A function `fME(f, x, h)` that computes the forward marginal effect using a vector of stepsizes h.
- A function `NLM(f, x, h, t_vals)` that computes the Non-Linearity Measure as described in d) where `t_vals` are the grid points used to estimate the NLM.

Use them to confirm the results from sub-tasks c) and d).

Exercise 3: ALE

Learning goals

1. Explain how ALE uses intervals and local prediction differences to estimate feature effects.
2. Implement the ALE algorithm (interval bounds, accumulation, centering) from scratch.
3. Prepare and visualize ALE curves for a fitted model.

Accumulated local effects describe how features influence the prediction of a machine learning model on average. The following tasks guide you to implement ALE from scratch and to test your code on the wheat seeds dataset (data/wheat_seeds.csv). We prepare the data and fit a random forest; your functions are then tested on this model:

R

```r
## wheat seeds data: predict the (normalized) Type from the numeric features
df = na.omit(read.csv("../data/wheat_seeds.csv"))
df$Type = (df$Type - min(df$Type)) / (max(df$Type) - min(df$Type))
X = df[, setdiff(names(df), "Type")]

## fit a random forest on a 70% training split
set.seed(42)
train = sample(nrow(X), 0.7 * nrow(X))
model = randomForest(x = X[train, ], y = df$Type[train])
```

Python

```python
# wheat seeds data: predict the (normalized) Type from the numeric features
df = pd.read_csv("../data/wheat_seeds.csv").dropna()
y = (df["Type"] - df["Type"].min()) / (df["Type"].max() - df["Type"].min())
X = df.drop(columns="Type").to_numpy()
feature_names = list(df.drop(columns="Type").columns)

# fit a random forest on a 70% training split
rng = np.random.default_rng(42)
train = rng.choice(len(X), size=int(0.7 * len(X)), replace=False)
model = RandomForestRegressor(random_state=42).fit(X[train], y.to_numpy()[train])
```

a) Since ALE works with bins, we have to select bounds first. Usually the bins are chosen s.t. every bin has roughly the same number of data points. However, to make it more simple, we use the same length for every interval. Complete the function `get_bounds()` to select bounds between the minimum and maximum value of a feature by using `n_intervals`. Those bounds are used for the calculation of ALE in the next tasks.

b) Implement the ALE algorithm from scratch inside the function `calculate_ale()` using the bounds from the previous task and the help of the lecture material. For each interval `k`:

- Select the observations inside the interval (if zero observations are found then return 0 for this interval). In general x_s can be counted as included if it is bigger than the lower bound and smaller or equal than the upper bound. However, make sure the elements in the first bound are included as well.
- Intervention: Use the relevant observations to create two new datasets:
  - `X_min`: Values at s-th feature position are replaced by the lower interval value z_{k-1}.

  - `X_max`: Values at s-th feature position are replaced by the higher interval value z_k.
- Prediction: Get the predictions for both `X_min` and `X_max`. Compute the differences in predictions of `X_max` to `X_min` and average them.

Use `cumsum` in R or `np.cumsum` in Python to perform the aggregation step and therefore to get accumulated values for each interval value. Finally, if the `center` parameter is set, center each interval by subtracting the average of all interval values.

Hint

`mapply` in R or `zip(bounds, bounds[1:])` in Python are nice tricks to iterate over the lower and upper bounds at the same time.

c) The function `calculate_ale()` returns both the used bounds and the ALE data. To use those data with `ggplot2` (R) or `matplotlib` (Python), you have to get the corresponding interval centers. Implement the function `prepare_ale()`, which should call `calculate_ale()` first and then compute the averages/centers per interval. In the end `prepare_ale()` should return the centers plus the corresponding ALE values – either as a `data.frame` (R) or two vectors (Python).

Exercise 4: Categorical ALE

Learning goals

1. Interpret PDPs and ALE plots for categorical features in a risk-modelling context.
2. Explain when ALE plots are preferable to PDPs.
3. Implement the distributional distance for categorical features used to order the levels of a nominal feature.

In this exercise, we use the German Credit dataset (data/credit.csv) and a random forest to predict whether a customer is a high or low risk for a bank. An overview of the dataset is given below:

	name	type	mean	nlevs
1	credit_risk	factor		2
2	age	integer	35.55	0
3	amount	integer	3271.26	0
4	credit_history	factor		5
5	duration	integer	20.90	0
6	employment_duration	factor		5
7	personal_status_sex	factor		4
8	purpose	factor		10

Since the decision of whether a person gets a loan can have serious implications on the person's life, banks are subordinate to regulations and must disclose the underlying mechanism of their used model to authorities. Since looking on the single trees is not feasible to uncover the internals of a random forest, (model-agnostic) interpretation methods should help to unfold the underlying mechanisms and to explain specific decisions.

a) Since the regulations require that the model does not discriminate against certain demographic groups, you want to evaluate the feature effect of `personal_status_sex` of the fitted forest model. Your colleague provides the following PDP. Interpret the PDP.

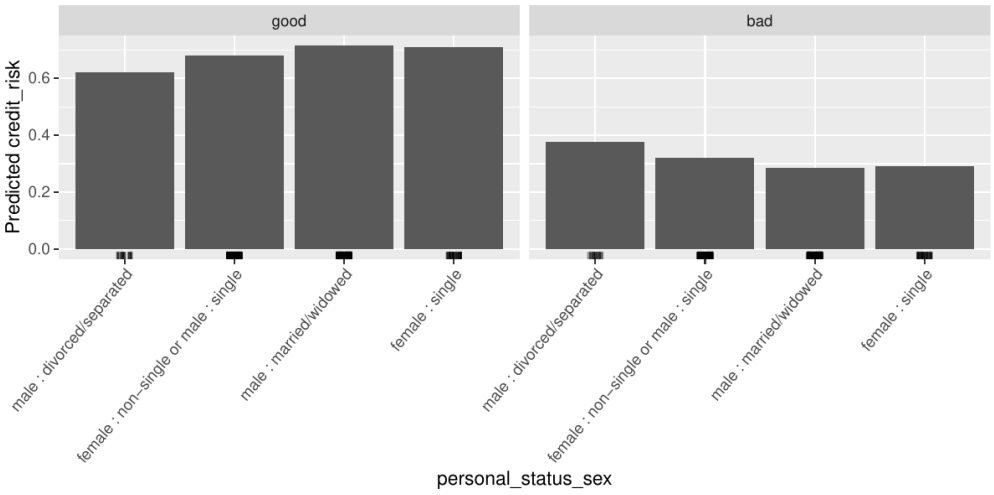


Figure 1: PDP of personal_status_sex

b) You also request an ALE plot from your colleague. Explain: Why and in which situations is inspecting ALE plots besides PDPs advisable?

c) Because the ALE model accumulates effects in a specific direction, the feature values must have an order by definition. However, there is no natural order to categorical/nominal

features like `personal_status_sex`. In order to derive an artificial order, Molnar 2022 (Chapter 8.2) proposes to order the categories of the feature x_s according to their similarity based on other/remaining features $x_c \in X_C$. Read the corresponding subsection in Molnar 2022.

In detail, the approach has the following steps:

(i) For each feature $x_c \in X_C$ do the following:
- Get the conditional empirical distributions of x_c given the categories of x_s: For numeric feature x_c, estimate the cumulative distribution function for each category of x_s separately and derive its values at predefined points (e.g., at deciles of x_c). For categorical feature x_c derive the relative frequency tables for each category of x_s separately.
- Compute the pairwise distances of distributions for each pair of categories of x_s: For numeric feature x_c, the distance is equal to the absolute maximum point-wise distance of the two empirical distribution functions. For categorical feature x_c, the distance is equal to the sum of the absolute difference of both relative frequency tables.

(ii) Sum up the pairwise distances of the distributions over features x_c in X_C for each class pair of x_s.

(iii) Reduce the resulting distance matrix to a single dimension using multi-dimensional scaling.

(iv) Order the categories of x_s according to the obtained similarity values.

Your colleague already implemented some parts of the algorithm, namely `order_levels()` and the distance of distributions for a **numeric** feature x_c in `get_diff_numeric()`:

R

```r
credit = read.csv("../data/credit.csv", stringsAsFactors = TRUE)

order_levels = function(data, feature.name) {
  feature = data[, feature.name]
  others = setdiff(colnames(data), feature.name)
  feature.lev = unique(feature)

  ### Iterate over other features
  dists = lapply(others, function(k) {
    feature.k = data[, k]
    if (inherits(feature.k, "factor") | inherits(feature.k, "character")) {
      dists = get_diff_cat(feature.k, feature)
    } else {
      dists = get_diff_numeric(feature.k, feature)
    }
    dists
  })
  dists.cumulated.long = as.data.frame(Reduce(function(d1, d2) {
    d1$dist = d1$dist + d2$dist
    d1
  }, dists))

  ### Create a matrix of distances
  dists.cumulated = reshape2::dcast(dists.cumulated.long,
    class1 ~ class2, value.var = "dist")[, -1]
  rownames(dists.cumulated) = colnames(dists.cumulated)

  ### conduct multi-dimensional scaling (here: principal coordinates analysis)
  ### based on dissimilarity matrix it assigns
  ### to each item a location in a low dimensional space
  ### the closer the location, the more similar the items are.
  scaled = cmdscale(dists.cumulated, k = 1)
  feature.lev[order(scaled)]
}

get_diff_numeric = function(feature.k, feature.j) {
  ### set up data.frame which we will fill later
  dists = expand.grid(unique(feature.j), unique(feature.j))
  colnames(dists) = c("class1", "class2")

  ### get decentiles
  quants = quantile(feature.k, probs = seq(0, 1, length.out = 10),
    na.rm = TRUE, names = FALSE)

  ### derive empirical distribution function for each category
  ecdfs = data.frame(lapply(unique(feature.j), function(lev) {
    x.ecdf = ecdf(feature.k[feature.j == lev])(quants)
    return(x.ecdf)
  }))
  colnames(ecdfs) = unique(feature.j)

  ### get pairwise absolute distances of empirical distribution function
  ### between the different categories
  ecdf.dists.all = abs(ecdfs[, dists$class1] - ecdfs[, dists$class2])

  ### get maximum distance over decentiles for each pair of categories
  dists$dist = apply(ecdf.dists.all, 2, max)
  dists
}
```

Python

```
credit = pd.read_csv("../data/credit.csv")


def order_levels(data, feature_name):
    """
    Orders levels of a nominal scaled feature according to other features in a dataset.

    Parameters:
        data (pd.DataFrame): Input data.
        feature_name (str): Name of the categorical feature which should be ordered.

    Returns: np.array (str) of ordered class labels
    """
    feature = data[feature_name].values
    others = list(data.columns)
    others.remove(feature_name)
    feature_lev = np.unique(feature)

    dists = []

    # Iterate over other features, sum up distances
    for i in range(len(others)):
        k = others[i]
        feature_k = data[k].values
        if (feature_k.dtype.kind != 'i') & (feature_k.dtype.kind != 'f'):
            ds = get_diff_cat(feature_k, feature)
        else:
            ds = get_diff_numeric(feature_k, feature)

        if i == 0:
            dists = ds
        else:
            dists['dist'] = dists['dist'] + ds['dist']

    # Create a matrix of distances
    dists = dists.pivot(index='class1', columns='class2', values='dist')

    # conduct classical multi-dimensional scaling (principal coordinates
    # analysis, like R's cmdscale): double-center the squared distance
    # matrix and project on the first principal coordinate
    D2 = np.asarray(dists, dtype=float) ** 2
    J = np.eye(len(D2)) - np.ones_like(D2) / len(D2)
    B = -0.5 * J @ D2 @ J
    eigval, eigvec = np.linalg.eigh(B)
    scaled = eigvec[:, -1] * np.sqrt(max(eigval[-1], 0))

    order = feature_lev[scaled.argsort()]

    return order


def get_diff_numeric(feature_k, feature_j):
    """
    Calculates the pairwise distances of classes of j of a numeric feature k.

    Parameters:
        feature_k (np.array): Values of the numeric feature.
        feature_j (np.array): Values of the categorical feature for which
            similarity based on feature k should be assessed.

    Returns: pd.DataFrame with three columns:
        class1: name of the first class
        class2: name of the second class
        dist: the distributional distance between the two classes for feature k
    """
```

Your colleague needs your help with implementing the computation of the distance of distribution for a **categorical** feature x_c in `get_diff_cat()`:

Hint

In R, the command `expand.grid(levels(feature.j), levels(feature.j))` creates the first two columns in your resulting dataset – the pairwise class combinations for x_s.

- Have a look on the functions `order_levels()` and `get_diff_numeric()`. Locate the steps of above's algorithmic description in the underlying code.
- Implement the missing rows in `get_diff_cat()`.
- Test your function with `personal_status_sex` as x_s and `employment_duration` as x_c.

Your returned result should look similar to this:

class1	class2	dist
male : married/widowed	male : married/widowed	0.00
female : non-single or male : single	male : married/widowed	0.22
male : divorced/separated	male : married/widowed	0.19
...	...	...

d) How did your function order the levels of `personal_status_sex`? Does this order make sense to you?

e) **Bonus:** Critically assess whether PDP/ALE alone replace a complete fairness audit.